

DSBDA Viva Theory Guide

Detailed theory for all practicals with code, examples, and intuition.

1. Data Wrangling I

Data Wrangling / Preprocessing — Why Bother?

Real-world data is never clean. You'll have missing cells, wrong data types, duplicate entries, inconsistent formatting, and values that make no sense. If you feed this directly into a model, it will fail or give garbage results. Data wrangling is the process of cleaning, transforming, and structuring raw data so it's ready for analysis.

Think of it like cooking: You don't cook with unwashed vegetables. Data wrangling = washing and chopping your ingredients.

DataFrame vs Series

- **DataFrame:** A 2D labeled data structure. Think of it as an Excel spreadsheet where each column can have a different type (numbers, text, dates). It's the primary object in pandas.
- **Series:** A single column extracted from a DataFrame. It's 1D with labels (index).

```
import pandas as pd
df = pd.read_csv("data.csv")      # df is a DataFrame
print(type(df))                  # <class 'pandas.core.frame.DataFrame'>

age_series = df["Age"]            # Extracting one column gives a Series
print(type(age_series))          # <class 'pandas.core.series.Series'>
```

You can think of a DataFrame as a dict of Series objects (one per column).

Missing Values — `isnull()`, `isna()`

Both functions do the exact same thing — they check every cell and return `True` if it's missing (NaN = Not a Number). Why two names? `isnull()` comes from the pandas/SQL convention (null = missing), `isna()` comes from numpy (NaN = Not a Number).

```
df.isnull().sum()                # Count missing per column - essential first step
df.isnull().sum().sum()          # Total missing values in entire dataset
```

Typical output when you run `df.isnull().sum():`

Age	177
Fare	0

Embarked	2
Survived	0

This tells you: Age has 177 missing values, Fare has none, Embarked has 2.

Handling Missing Values — Three Strategies

Method	What it does
Drop rows <code>dropna()</code>	Remove any row that has a NaN. Use when very few rows are affected (say < 5% of data).
Drop columns <code>drop(columns=[...])</code>	Remove an entire column if it has too many missing values (say > 50% missing).
Fill <code>fillna(value)</code>	Replace NaN with something — mean, median, mode, or a constant. Most common approach.

When to use what:

```
# If Age has 177 missing out of 891 (Titanic), dropping all those rows is wasteful
# Better to fill:
df["Age"].fillna(df["Age"].median(), inplace=True)

# If a column "Cabin" has 687 missing out of 891 – it's useless, drop it
df.drop("Cabin", axis=1, inplace=True)

# If only 2 rows have missing Embarked – just drop those rows
df.dropna(subset=["Embarked"], inplace=True)
```

Example — filling Age with mean:

```
Age column: [25, NaN, 30, NaN, 35]
Mean = (25 + 30 + 35) / 3 = 30
After fillna(30): [25, 30, 30, 30, 35]
```

`describe()`, `info()`, `shape`

`df.describe()` — generates summary statistics for ALL numeric columns at once:

	Age	Fare
count	714.000	891.000
mean	29.700	32.204
std	14.526	49.693
min	0.420	0.000
25%	20.125	7.910
50%	28.000	14.454

75%	38.000	31.000
max	80.000	512.329

From this table you can instantly spot: Age range (0.42 to 80), median age (28), Fare is right-skewed (mean 32 > median 14), Fare has extreme max (512).

df.info() — tells you column names, non-null counts, and data types:

```
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
PassengerId    891 non-null int64
Survived       891 non-null int64
Pclass         891 non-null int64
Name           891 non-null object
Sex            891 non-null object
Age            714 non-null float64  ← Only 714 non-null (177 missing)
Fare           891 non-null float64
Embarked       889 non-null object   ← 2 missing
```

This tells you immediately which columns need cleaning.

df.shape — returns (rows, columns). (891, 12) means 891 rows and 12 columns. Always check this after cleaning to confirm nothing went wrong.

Datatype Checking & Conversion

Models expect numbers. Strings ("25") must become integers (25). True/False must become 0/1.

```
df.dtypes          # Check every column's type
df["Age"] = df["Age"].astype("int")      # Convert float to int
df["Date"] = pd.to_datetime(df["Date"])  # Convert string to datetime
```

Common type issues:

- A numeric column stored as "object" (string) — often has some non-numeric values like "N/A" or "?" in it
- Boolean stored as string — "True" / "False" instead of True / False
- Dates stored as strings

Categorical vs Numerical Data

Type	What it is	Examples	Can you do math?
Numerical	Numbers with mathematical meaning	Age, Fare, Temperature, Salary	Yes

Type	What it is	Examples	Can you do math?
Categorical	Labels or groups	Gender, Color, City, Species	No (unless encoded)

Categorical data can be further split:

- **Nominal:** No natural order — Color (Red, Green, Blue), City (Mumbai, Delhi, Pune)
- **Ordinal:** Has order — Education (School < College < Masters), Size (Small < Medium < Large)

Label Encoding vs One-Hot Encoding

Label Encoding — assign each category a number (0, 1, 2, ...):

```
Gender: Male → 0, Female → 1, Other → 2
```

Problem: The model might think $2 > 1 > 0$, implying "Other" > "Female" > "Male". This is fine for ordinal data but dangerous for nominal.

One-Hot Encoding — create one binary column per category:

```
Gender_Male = 1 if Male else 0
Gender_Female = 1 if Female else 0
Gender_Other = 1 if Other else 0
```

Now there's no false ordering. But you get more columns (k columns for k categories).

Common trick: Drop the first column to avoid multicollinearity (k-1 columns instead of k):

```
pd.get_dummies(df["Color"], drop_first=True)
```

When to use which:

Situation	Use
Ordinal data (Education, Rating)	Label Encoding
Nominal data, small #categories (<10)	One-Hot Encoding
Nominal data, many categories (1000 cities)	Label Encoding (or target encoding)

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Gender_encoded"] = le.fit_transform(df["Gender"])
# Male → 0, Female → 1
```

```
# One-hot:  
df_encoded = pd.get_dummies(df, columns=["Color"])  
# Color_Red, Color_Green, Color_Blue columns created
```

Normalization vs Standardization

Both are **feature scaling** techniques. Many ML algorithms (SVM, logistic regression, KNN, neural networks) expect features to be on a similar scale. Without scaling, a feature like "Salary" (range 20,000–200,000) will dominate "Age" (range 20–80) even if Age is more important.

Normalization (Min-Max Scaling):

```
X_scaled = (X - X.min()) / (X.max() - X.min())
```

- Output range: [0, 1]
- **Sensitive to outliers** — one extreme value compresses everything else
- Use when: data doesn't have outliers, or you need bounded output

Standardization (Z-score Scaling):

```
X_scaled = (X - X.mean()) / X.std()
```

- Output: centered at 0, standard deviation = 1
- Range: unbounded (typically -3 to +3 for normal data)
- **Robust to outliers** (they just get large z-scores but don't compress other values)
- Use when: data has outliers, or algorithm assumes normal distribution

Example — Salary column [30000, 40000, 50000, 200000]:

- Normalization: [0, 0.059, 0.118, 1.0] — the first three values are compressed near 0
- Standardization: [-0.63, -0.56, -0.48, 1.67] — better spread

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler  
minmax = MinMaxScaler()  
standard = StandardScaler()  
  
X_norm = minmax.fit_transform(X)    # Normalize  
X_std = standard.fit_transform(X)   # Standardize
```

2. Data Wrangling II

Missing Values & Inconsistent Data

Covered the basics in Practical 1. Additional issues you'll encounter:

Inconsistent categorical data — the same category written multiple ways:

"Male", "male", "M", "m" → all mean the same but look different to Python
 "US", "U.S.", "USA", "America" → same country, different strings

Fix with string operations:

```
df["Gender"] = df["Gender"].str.lower().str.strip()
df["Gender"] = df["Gender"].map({"male": "Male", "female": "Female", "m": "Male",
"f": "Female"})
```

Or with a function:

```
def clean_gender(val):
    val = str(val).lower().strip()
    if val in ["m", "male"]:
        return "Male"
    elif val in ["f", "female"]:
        return "Female"
    else:
        return "Other"

df["Gender"] = df["Gender"].apply(clean_gender)
```

Duplicate Data

Duplicates are exact copies of a row. They bias your analysis by giving more weight to those observations.

```
df.duplicated()           # Returns True/False per row
df.duplicated().sum()     # Count total duplicates
df[df.duplicated()]       # Show duplicate rows
df.drop_duplicates()      # Remove duplicates (keeps first occurrence)
df.drop_duplicates(keep="last") # Keep last occurrence
```

Invalid Values

These are values that are technically present (not NaN) but don't make sense:

- Age = -5 (negative)
- Temperature = 500°C (impossible for weather)
- Salary = 0 for an employed person
- Gender = "Banana"

Detect by domain knowledge + descriptive stats:

```
df.describe()           # Check min/max — is Age.min() negative?
df["Age"].value_counts() # Check unique values — anything weird?
```

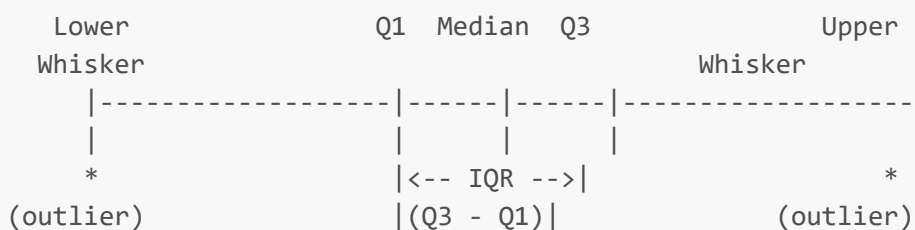
Fix by replacing with NaN then imputing, or capping to valid range:

```
df.loc[df["Age"] < 0, "Age"] = df["Age"].median()
```

Outliers — What Are They?

Outliers are values significantly different from the rest. An outlier IS NOT necessarily an error — it could be a genuine rare event (a billionaire in a salary dataset). But outliers can distort statistical measures and model performance.

Boxplot Anatomy — Diagram with Labels



- **Q1 (25th percentile)**: 25% of data is below this line
- **Median (Q2, 50th percentile)**: Middle of data — 50% below, 50% above
- **Q3 (75th percentile)**: 75% of data is below this line
- **IQR = Q3 - Q1**: The middle 50% of your data. A measure of spread that ignores extremes
- **Lower Whisker**: $Q1 - 1.5 \times IQR$ (any point below this is an outlier)
- **Upper Whisker**: $Q3 + 1.5 \times IQR$ (any point above this is an outlier)
- **Outliers**: Individual points plotted beyond the whiskers

Why 1.5? It's a convention from John Tukey. For a normal distribution, about 99.3% of data falls within $1.5 \times IQR$. Change to $3 \times IQR$ for "extreme outliers" only.

Outlier Removal using IQR — Step by Step

```
# 1. Calculate quartiles
Q1 = df["Age"].quantile(0.25)
Q3 = df["Age"].quantile(0.75)

# 2. Calculate IQR
IQR = Q3 - Q1

# 3. Calculate bounds
```

```

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# 4. Filter out outliers
df_clean = df[(df["Age"] >= lower_bound) & (df["Age"] <= upper_bound)]

# 5. Check how many were removed
print(f"Removed {len(df) - len(df_clean)} outlier rows")

```

Example with actual numbers — Age column:

```

Q1 = 20.125, Q3 = 38.000
IQR = 38 - 20.125 = 17.875
Lower = 20.125 - 1.5 × 17.875 = 20.125 - 26.8125 = -6.6875
Upper = 38 + 1.5 × 17.875 = 38 + 26.8125 = 64.8125

```

Any age below -6.7 (none) or above 64.8 (a few) would be outliers.

Skewness — Full Picture

Skewness measures how asymmetric a distribution is:

Positive (Right) Skew:

```

      *
     ***
    *****
   *********
  -----

```

Long tail on the right. Mean > Median (because the tail pulls the mean right).

- **Examples:** Income, house prices, insurance claims — most people earn ~30K, a few earn millions
- **Real Titanic fare data:** Most tickets were cheap (under 50), a few were very expensive (>500)

Negative (Left) Skew:

```

*****
*****
  ***
   *
  -----

```

Long tail on the left. Mean < Median.

- **Examples:** Age at death (most die old, few die young), exam scores if test is easy

Zero (Symmetric):

```

***
*****
***
-----

```

Normal distribution. Mean $\approx$ Median.

How to check skewness numerically:

```
df["Fare"].skew() # For Titanic, this gives something like 4.7 (highly right-skewed)
```

How does skewness affect analysis?

- Many statistical tests assume normality (t-test, ANOVA)
- Mean is misleading for skewed data (a few billionaires make "average income" look high)
- Regression models work better with normally distributed targets

Data Transformation — Fixing Skewness**Log transform (`np.log(x)`):**

- Best for strong positive skew
- Compresses large values more than small values
- Can't handle zeros or negatives (`np.log1p(x)` handles zeros by computing `log(1+x)`)
- Interpretability: a 1-unit increase in `log(price)` $\approx$ 1% increase in price

Square-root transform (`np.sqrt(x)`):

- Weaker than log transform
- Good for moderate positive skew (count data, e.g., number of transactions)
- Handles zeros fine

When to use which:

```

Skewness < 0.5 → No transform needed
0.5 < Skewness < 1 → Square-root
Skewness > 1 → Log transform

```

```

import numpy as np
df["Fare_log"] = np.log1p(df["Fare"]) # log(1 + Fare)
df["Fare_sqrt"] = np.sqrt(df["Fare"])

```

```
# Check if skewness improved
print("Original skew:", df["Fare"].skew())
print("After log:", df["Fare_log"].skew())
print("After sqrt:", df["Fare_sqrt"].skew())
```

Min-Max vs Z-Score (Recap with Examples)

Scenario	Pick this
Neural networks need input in [0,1]	Min-Max
Data has no outliers, small range	Min-Max
Data has outliers	Z-score
You want to compare values across different scales	Z-score
Image pixels (always 0-255)	Min-Max (to [0,1])

3. Descriptive Statistics

Measures of Central Tendency

These tell you where the "center" or "typical value" of your data lies.

Mean (Average):

```
mean = df["Age"].mean()
```

Sum of all values divided by count. **Pros:** Uses all data points. **Cons:** Sensitive to outliers.

Example — Salaries [30K, 35K, 40K, 45K, 500K]:

- Mean = $(30+35+40+45+500)/5 = 130K$
- The one billionaire makes it look like everyone earns 130K. Misleading!

Median:

```
median = df["Age"].median()
```

Middle value after sorting. For [30, 35, 40, 45, 500], median = **40K** (much more representative).

Mode:

```
mode = df["Embarked"].mode()[0] # Most common embarkation port
```

Most frequent value. Only measure that works for categorical data. A distribution can have multiple modes (bimodal = two peaks).

Measures of Dispersion

These tell you how spread out your data is.

Measure	What it tells you	Sensitive to outliers?
Range = Max - Min	Total spread	Yes (one extreme changes it completely)
Variance = $\sum (x - \mu)^2 / n$	Average squared distance from mean	Yes (squaring magnifies outliers)
Std Dev = $\sqrt{\text{Variance}}$	Spread in original units	Yes
IQR = Q3 - Q1	Spread of middle 50%	No (ignores extremes)

Concrete example — Ages [20, 22, 24, 26, 28, 30, 100]:

```
Mean = 35.7
Range = 100 - 20 = 80
Variance = [(20-35.7)2 + ... + (100-35.7)2] / 7 ≈ 689
Std Dev =  $\sqrt{689} \approx 26.2$ 
Q1 = 22, Q3 = 30, IQR = 8
```

Notice: IQR (8) gives a much better sense of where most ages are than std dev (26.2) which is inflated by the 100.

Percentiles & Quartiles

- **25th percentile (Q1):** 25% of values are below this
- **50th percentile (Q2 = Median):** 50% below
- **75th percentile (Q3):** 75% below
- **90th percentile:** 90% below (only top 10% above)

Example — exam scores [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]:

- 25th percentile = 30 (25% scored $\leq$ 30)
- Median = 55
- 75th percentile = 80

```
df["Age"].quantile([0.25, 0.5, 0.75, 0.9])
```

Grouped Statistics — `groupby()` + `agg()`

This is one of the most practical pandas operations. You split data into groups, apply a function to each group, and combine results.

Without groupby: You get one statistic for the whole dataset. **With groupby:** You get statistics PER CATEGORY.

Example — Iris dataset, petal length statistics per species:

```
df.groupby("Species")["PetalLength"].agg(["mean", "median", "std", "min", "max"])
```

Output:

	mean	median	std	min	max
Species					
setosa	1.462	1.50	0.174	1.0	1.9
versicolor	4.260	4.35	0.469	3.0	5.1
virginica	5.552	5.55	0.552	4.5	6.9

This tells you: Setosa has small petals (~1.5), virginica has large petals (~5.6), versicolor is in between (~4.3). This is the kind of insight that helps you build classifiers.

Multiple aggregations on multiple columns:

```
df.groupby("Species").agg({
    "PetalLength": ["mean", "std"],
    "SepalLength": ["mean", "std"],
    "PetalWidth": ["mean", "std"]
})
```

Group by multiple categories:

```
titanic.groupby(["Pclass", "Sex"])["Fare"].mean()
# Average fare paid by men in 1st class, women in 1st class, etc.
```

Categorical vs Quantitative Variable — Deeper

- **Quantitative:** You can do arithmetic. Mean age, total fare, max temperature. These are your dependent variables in regression.
- **Categorical:** You count frequencies. "70% of passengers were male." These become your groups in groupby.

Iris Dataset — Why is it famous?

- Created by R.A. Fisher in 1936 (one of the first datasets for ML)

- 3 species of iris flowers, 50 samples each
- 4 measurements: sepal length, sepal width, petal length, petal width
- Key property: **one species (setosa) is linearly separable** from the other two. This makes it a perfect teaching dataset.
- Real application: After training on measurements of known species, you can identify an unknown iris flower.

The challenge: Versicolor and virginica overlap in their feature ranges — no single measurement separates them perfectly.

4. Linear Regression

Supervised Learning — The Big Picture

You have input data (features X) and you know the correct answer (label y). The model learns to map $X \rightarrow y$ by finding patterns.

- **Training:** Show the model ($X_{\text{train}}, y_{\text{train}}$) pairs. It adjusts its parameters to minimize error.
- **Testing:** Give it X_{test} (no y). It predicts y_{pred} . Compare y_{pred} vs y_{test} to measure performance.

Analogy: Like studying with a answer key (training) then taking an exam without answers (testing).

Regression vs Classification — One Simple Distinction

- **Regression answers "How much?"** — house price, temperature, salary, stock price
- **Classification answers "Which category?"** — spam/not spam, cat/dog, species

Linear Regression — Intuition

You want to draw the best straight line through your data points.

Simple linear regression (one feature):

$$y = m \times x + c$$

- x = input feature (e.g., number of rooms)
- y = target (e.g., house price)
- m = slope/coefficient (how much y changes when x increases by 1)
- c = intercept (predicted y when $x = 0$)

Example — predicting price from number of rooms:

```
Price = 50 × Rooms + 20
If Rooms = 3: Price = 50 × 3 + 20 = 170 (lakhs)
If Rooms = 5: Price = 50 × 5 + 20 = 270
```

Here, each additional room adds 50 lakhs.

Multiple Linear Regression (many features)

$$y = m_1x_1 + m_2x_2 + \dots + m_nx_n + c$$

Boston Housing example:

$$\text{Price} = -0.01(\text{CRIM}) + 3.8(\text{RM}) + 0.01(\text{DIS}) - 0.5(\text{AGE}) + \dots + c$$

- CRIM (crime rate): negative coefficient → higher crime = lower price
- RM (rooms): positive → more rooms = higher price
- The coefficient tells you: "for 1 unit increase in this feature, price changes by this much, holding all else constant"

How Does the Model Find the Best Line?

Ordinary Least Squares (OLS): The model tries all possible lines and picks the one that minimizes the sum of squared errors (the distance between actual points and predicted points, squared).

Error = Actual - Predicted
Goal: Minimize $\sum(\text{Error})^2 = \sum(y_{\text{actual}} - y_{\text{pred}})^2$

Why square? So positive and negative errors don't cancel out, and large errors are penalized more.

Correlation & Correlation Matrix

- **Correlation coefficient (r):** Measures strength AND direction of LINEAR relationship between two variables
 - $r = +1$: Perfect positive (x up, y up)
 - $r = -1$: Perfect negative (x up, y down)
 - $r = 0$: No linear relationship
- **Correlation matrix:** Shows correlations between ALL pairs of variables

```
df.corr() # Returns a table where cell (i,j) = correlation between column i and j
```

Example — Boston Housing correlation with PRICE:

Feature	Correlation with PRICE	
RM	+0.70	← Strong positive (more rooms = higher price)
LSTAT	-0.74	← Strong negative (more low-status = lower price)

AGE	-0.38	← Moderate negative
CRIM	-0.39	← Moderate negative

Important: Correlation $\neq$ Causation. Ice cream sales and drowning are correlated (both happen in summer), but buying ice cream doesn't cause drowning.

Train-Test Split — Why Random?

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

- **test_size=0.2:** 80% for training, 20% for testing (common split)
- **random_state=42:** Ensures reproducibility — same "random" split every time
- **Why split?** If you test on the same data you trained on, the model just memorizes (overfitting). It would look perfect but fail on new data.

The cardinal rule of ML: Never let the model see test data during training.

Prediction & Residual

```
y_pred = model.predict(X_test)
residuals = y_test - y_pred
```

A residual is the error for a single prediction:

- Positive residual = model under-predicted (actual > predicted)
- Negative residual = model over-predicted (predicted > actual)

Good models have residuals centered around 0 with no pattern. If residuals show a pattern (e.g., increasing with predicted value), the model is missing something (non-linearity).

Evaluation Metrics — Which One Matters?

Metric	What it measures	Range	Notes
MAE	Average absolute error	$[0, \infty)$	Easy to explain: "on average, predictions are off by \$X"
MSE	Average squared error	$[0, \infty)$	Punishes large errors. Higher than MAE.
RMSE	Square root of MSE	$[0, \infty)$	Same unit as target. Most commonly reported.
R²	Proportion of variance explained	$(-\infty, 1]$	1 = perfect, 0 = as good as predicting mean, negative = worse than mean

Example — predicting house prices:

```
MAE = $15,000    ← On average, predictions are off by $15K
RMSE = $22,000   ← Larger errors are penalized more
R2 = 0.85       ← Model explains 85% of price variation
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

Outlier Effect on Regression

Outliers pull the regression line toward themselves. This is because OLS minimizes squared error — a point far from the line contributes a huge squared error, so the line bends toward it.

```
Without outlier:  *--*--*--*--*--*--* (line passes through data)
With outlier:     *--*--*--*--*--*--* (line tilted toward outlier)
                  ↑
                  (outlier far to right)
```

Result: Lower R^2 , biased coefficients, unreliable predictions.

5. Logistic Regression

Classification vs Binary Classification

- **Classification:** Predicting a discrete class label
- **Binary classification:** Exactly two classes — Spam/Not Spam, Buy/Not Buy, Survived/Died, Disease/No Disease

Why Not Linear Regression for Classification?

Linear regression can output any number ($-\infty$ to $+\infty$). For binary classification we need a probability $[0, 1]$. Logistic regression solves this by wrapping linear output in the sigmoid function.

The Sigmoid Function — The Key Ingredient

```
Step 1: Compute  $z = m_1x_1 + m_2x_2 + \dots + c$  (same as linear regression)
Step 2: Convert  $z$  to probability:  $p = 1 / (1 + e^{-z})$ 
```

The sigmoid squashes any real number to $[0, 1]$:


```

z = -10 → p = 0.000045 (very unlikely)
z = -2  → p = 0.12      (unlikely)
z = 0   → p = 0.50      (exactly 50-50)
z = +2  → p = 0.88      (likely)
z = +10 → p = 0.999955 (almost certain)

```

Threshold & Decision Boundary

- By default, threshold = 0.5
 - $p \geq 0.5 \rightarrow$ Class 1 (e.g., "Will buy")
 - $p < 0.5 \rightarrow$ Class 0 (e.g., "Won't buy")

Decision boundary — the line where $p = 0.5$ (which is when $z = 0$):

$$m_1x_1 + m_2x_2 + \dots + c = 0$$

On one side of this line: predict class 1. On the other: predict class 0.

Feature Scaling — Absolutely Necessary for Logistic Regression

Logistic regression uses gradient descent to find optimal coefficients. If features are on very different scales (Age: 20-80 vs Salary: 30K-200K), gradient descent zigzags slowly toward the minimum.

```

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# Fit on TRAINING data only
X_train_scaled = scaler.fit_transform(X_train)

# Transform TEST data using the SAME scaler (don't fit again!)
X_test_scaled = scaler.transform(X_test)

```

Why `fit_transform` on train but only `transform` on test? The scaler learns mean and std from training data. Using test data to compute scaling would mean test data influences training — that's data leakage.

Confusion Matrix — Visualizing Classification Performance

	Actual Values	
	Positive	Negative
Predicted Pos	TP	FP
Predicted Neg	FN	TN

Example — disease detection (100 patients, 10 actually have disease):

	Actual: Sick	Actual: Healthy
Predicted: Sick	8	5
Predicted: Healthy	2	85

- TP = 8 (correctly identified sick patients)
- TN = 85 (correctly identified healthy)
- FP = 5 (healthy people told they're sick — causes unnecessary stress)
- FN = 2 (sick people told they're healthy — dangerous!)

Evaluation Metrics — Intuition

Metric	Formula	When to care about it
Accuracy	$(TP+TN)/(TP+TN+FP+FN) = 93\%$	Good when classes are balanced
Precision	$TP/(TP+FP) = 8/13 = 61.5\%$	When false positives are costly (spam detection — you don't want to lose important emails)
Recall	$TP/(TP+FN) = 8/10 = 80\%$	When false negatives are costly (disease detection — you don't want to miss a sick patient)
F1-Score	$2 \times P \times R / (P + R) \approx 69.6\%$	Harmonic mean — balances precision and recall

Trade-off: You can always get 100% recall by predicting everyone as sick (TP = all sick, FP = all healthy → precision drops). You can get 100% precision by only predicting the most certain cases (TP = few, FN = many → recall drops). F1 finds a middle ground.

```
from sklearn.metrics import confusion_matrix, classification_report

cm = confusion_matrix(y_test, y_pred)
# cm = [[TN, FP], [FN, TP]] ← Note the order!

print(classification_report(y_test, y_pred))
# Gives precision, recall, f1 for each class
```

Social Network Ads Dataset

- **Rows:** 400 social network users
- **Features:** Age, Estimated Salary
- **Target:** Purchased (0 = didn't buy, 1 = bought the product)
- **Goal:** Can we predict who will buy based on age and salary?

The non-linear decision boundary makes this interesting — young high-earners and older medium-earners might be the buyers, but the pattern isn't a simple straight line.

6. Naive Bayes

Bayes Theorem — The Foundation

$$P(A|B) = P(B|A) \times P(A) / P(B)$$

Read as: "Probability of A given B equals probability of B given A times prior probability of A, divided by evidence."

Real-world intuition — medical testing:

- A = You have a disease
- B = You tested positive
- P(A) = Prior: 1% of population has the disease
- P(B|A) = Likelihood: Test is 99% accurate for sick people
- P(B) = Evidence: 2% of all tests come back positive

$$P(\text{Disease} \mid \text{Positive}) = 0.99 \times 0.01 / 0.02 = 0.495$$

Even with a 99% accurate test, there's only ~50% chance you actually have the disease if you test positive. This counter-intuitive result (base rate fallacy) is why Bayes theorem matters.

Naive Bayes Classifier

Applies Bayes theorem to classification:

$$P(\text{Class} \mid \text{Features}) = [P(\text{Features} \mid \text{Class}) \times P(\text{Class})] / P(\text{Features})$$

Since P(Features) is constant for all classes, we can simplify:

$$P(\text{Class} \mid \text{Features}) \propto P(\text{Class}) \times P(\text{Feature}_1 \mid \text{Class}) \times P(\text{Feature}_2 \mid \text{Class}) \times \dots$$

The "Naive" Assumption: All features are INDEPENDENT given the class. This means we can multiply probabilities. This assumption is almost always wrong in real life (petal length and petal width are correlated), but the algorithm still works surprisingly well.

Why does it work despite being wrong? Because even if the probability estimates are off, the relative ranking of classes is usually correct. It's like a poorly calibrated but still useful classifier.

Full Example — Iris Classification

Say we have an iris with petal length = 5.0 and we want to classify it.

Step 1: Prior probabilities

```
P(setosa) = 50/150 = 0.333  
P(versicolor) = 50/150 = 0.333  
P(virginica) = 50/150 = 0.333
```

(Equal priors since each species has 50 samples.)

Step 2: Likelihood — probability of seeing petal length = 5.0 given each species

For Gaussian Naive Bayes, we assume each feature follows a normal distribution within each class. We compute the probability density:

```
from scipy.stats import norm  
# For versicolor: mean=4.26, std=0.47  
likelihood_versicolor = norm.pdf(5.0, loc=4.26, scale=0.47)  
  
# For virginica: mean=5.55, std=0.55  
likelihood_virginica = norm.pdf(5.0, loc=5.55, scale=0.55)
```

Step 3: Posterior (unnormalized)

```
setosa_score = 0.333 × P(5.0 | setosa) ≈ 0 (very unlikely – setosa has small  
petals)  
versicolor_score = 0.333 × P(5.0 | versicolor) ≈ 0.20  
virginica_score = 0.333 × P(5.0 | virginica) ≈ 0.35
```

Step 4: Predict the class with highest posterior = virginica

Gaussian Naive Bayes

Best for continuous features that roughly follow a normal distribution within each class.

```
from sklearn.naive_bayes import GaussianNB  
model = GaussianNB()  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
  
# Check class priors learned by model  
print(model.class_prior_)      # P(setosa), P(versicolor), P(virginica)
```

Multiclass Classification

Unlike logistic regression (natively binary), Naive Bayes handles multiple classes naturally — just compute posterior for each class and pick the largest.

Use cases:

- Spam filtering (spam / promotions / primary)
- News categorization (sports / politics / tech / entertainment)
- Iris classification (setosa / versicolor / virginica)

Prior vs Posterior — In Simple Terms

- **Prior:** What you believe before seeing evidence. "10% of all emails are spam."
- **Posterior:** What you believe after seeing evidence. "This email contains the word 'lottery', so there's a 90% chance it's spam."

Why Naive Bayes Is Popular

- Fast to train and predict (just multiply probabilities)
- Works with small datasets
- Handles high-dimensional data well (text classification with thousands of features)
- Gives probability estimates (not just class labels)

Evaluation

Same metrics as logistic regression: confusion matrix, accuracy, precision, recall, F1. The confusion matrix interpretation is identical.

7. Text Analytics

NLP — Why Is Text Hard?

Computers understand numbers, not words. "Apple" could be a fruit or a company. "Run" could be a verb or a noun. Sarcasm, slang, typos, and context make text messy. NLP bridges the gap between human language and machine understanding.

Corpus & Document

- **Document:** One unit of text — a sentence, a tweet, a paragraph, an article, a book chapter
- **Corpus:** Collection of all documents you're working with (plural: corpora)

Analogy: A document is a single book. The corpus is your entire library.

Tokenization — Breaking Text Apart

```
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Dr. Smith went to N.Y.C. He bought 2 apples."

# Sentence tokenization
sentences = sent_tokenize(text)
# → ["Dr. Smith went to N.Y.C.", "He bought 2 apples."]

# Word tokenization
```

```
words = word_tokenize(text)
# → ["Dr.", "Smith", "went", "to", "N.Y.C.", "He", "bought", "2", "apples", "."]
```

Notice: "Dr." is kept as one token (not split into "Dr" and "."). "N.Y.C." is one token. This is smarter than just splitting by spaces.

Stop Words — Removing Noise

Stop words are very common words that don't add much meaning to text analysis. Removing them reduces dimensionality and improves model performance.

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
print(len(stop_words))          # ~179 words
print(list(stop_words)[:10])    # ['i', 'me', 'my', 'myself', 'we', ...]

sentence = "The movie was not good and I did not enjoy it"
words = word_tokenize(sentence)
filtered = [w for w in words if w.lower() not in stop_words]
# → ["movie", "good", "enjoy"]
```

Common stop words: a, an, the, is, was, were, I, you, he, she, it, we, they, and, or, but, in, on, at, to, for, of, with, this, that, these, those

Regular Expressions — Pattern Matching

Regex is a language for finding patterns in text. Essential for cleaning text data.

```
import re

text = "Contact: john@email.com, Call: +91-9876543210"

# Find all emails
emails = re.findall(r"\S+@\S+", text)          # → ["john@email.com"]

# Find all phone numbers
phones = re.findall(r"\+\d{2}-\d{10}", text)    # → ["+91-9876543210"]

# Remove all non-alphabetic characters
clean = re.sub(r"[^a-zA-Z ]", "", text)        # → "Contact john email com Call"

# Replace multiple spaces with single space
clean = re.sub(r"\s+", " ", clean).strip()     # → "Contact john email com Call"
```

Common regex patterns for viva:

- `\d+` — one or more digits

- `\w+` — one or more word characters (letters, digits, underscore)
- `\s+` — one or more whitespace characters
- `[^a-zA-Z]` — anything that is NOT a letter
- `^` — start of string, `$` — end of string

Stemming vs Lemmatization — The Key Difference

Both reduce words to their base form, but differently:

Aspect	Stemming	Lemmatization
Method	Chop off affixes based on rules	Use dictionary (WordNet) to find root
Speed	Fast	Slower
Output	May not be a real word	Always a real word
Example	"running" → "run", "flies" → "fli", "studies" → "studi"	"running" → "run", "flies" → "fly", "studies" → "study"
Library	<code>PorterStemmer</code> , <code>LancasterStemmer</code>	<code>WordNetLemmatizer</code>

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
ps = PorterStemmer()
lemmatizer = WordNetLemmatizer()

words = ["running", "flies", "better", "studies", "eating"]
for w in words:
    print(f"{w:10} → stem: {ps.stem(w):10} lemma: {lemmatizer.lemmatize(w, pos='v')}")
```

Output:

```
running    → stem: run      lemma: run
flies      → stem: fli     lemma: fly    ← stemming loses the 'y'
better     → stem: better   lemma: good   ← stemming keeps 'better' as is
studies    → stem: studi  lemma: study  ← stemming gives non-word
eating     → stem: eat     lemma: eat
```

When to use which? Use stemming for quick-and-dirty (search engines). Use lemmatization when you need correct words (chatbots, text generation).

POS Tagging — Grammar for Machines

Part-of-Speech tagging labels each word with its grammatical role.

```
from nltk import pos_tag
from nltk.tokenize import word_tokenize
```

```
text = "The cat sat on the mat"
tokens = word_tokenize(text)
tagged = pos_tag(tokens)
# → [('The', 'DT'), ('cat', 'NN'), ('sat', 'VBD'), ('on', 'IN'), ('the', 'DT'),
('mat', 'NN')]
```

Common POS tags:

Tag	Meaning	Examples
NN	Noun, singular	cat, dog, table
NNS	Noun, plural	cats, dogs
VB	Verb, base form	run, eat, be
VBD	Verb, past tense	ran, ate, was
VBG	Verb, gerund	running, eating
JJ	Adjective	big, red, beautiful
RB	Adverb	quickly, very, well
DT	Determiner	the, a, an, this
IN	Preposition	in, on, at, for

Why POS tagging matters: It disambiguates words. "Run" as a verb (NN vs VB changes how you process it). Lemmatization uses POS to get correct root ("running" depends on whether it's a verb or adjective).

TF-IDF — Term Frequency Inverse Document Frequency

The problem with simple word counts: common words like "the", "is" dominate even though they're meaningless. TF-IDF gives high weight to words that are frequent in one document but rare across all documents.

TF (Term Frequency):

$$TF(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total words in } d)$$

- A word that appears 3 times in a 100-word doc has $TF = 0.03$

IDF (Inverse Document Frequency):

$$IDF(t) = \log(\text{Total documents} / \text{Number of documents containing } t)$$

- A word that appears in all docs (e.g., "the") → $IDF = \log(1) = 0$
- A word that appears in 2 out of 100 docs → $IDF = \log(100/2) = \log(50) \approx 3.91$

TF-IDF = TF × IDF

Example — 3 documents about food:

```
Doc1: "Pizza is good. I love pizza."  
Doc2: "Pasta is good. I love pasta."  
Doc3: "Pizza is better than pasta."
```

- "pizza": appears in Doc1 (2 times) and Doc3 (1 time). Not in Doc2.
 - TF in Doc1 = $2/7 \approx 0.29$, IDF = $\log(3/2) \approx 0.41$, TF-IDF ≈ 0.12
 - TF in Doc3 = $1/5 = 0.2$, TF-IDF ≈ 0.082
 - TF in Doc2 = 0
- "good": appears in Doc1 and Doc2.
 - IDF = $\log(3/2) \approx 0.41$, but it appears only once in each doc
 - Moderate TF-IDF
- "is": appears in all 3 documents
 - IDF = $\log(3/3) = 0$, so TF-IDF = 0 regardless of TF

Result: "Pizza" and "Pasta" get high TF-IDF in their respective documents. "Is" gets zero. The model focuses on meaningful words.

```
from sklearn.feature_extraction.text import TfidfVectorizer  
  
docs = [  
    "Pizza is good. I love pizza.",  
    "Pasta is good. I love pasta.",  
    "Pizza is better than pasta."  
]  
  
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(docs)  
print(vectorizer.get_feature_names_out())  
# → ['better', 'good', 'is', 'love', 'pasta', 'pizza', 'than']  
print(X.toarray())  
# Shows TF-IDF scores for each word in each document
```

Bag of Words (BoW) — Simpler Alternative

Just counts words, ignores word order and importance. Each document becomes a vector of word counts.

```
from sklearn.feature_extraction.text import CountVectorizer  
  
cv = CountVectorizer()
```

```
X = cv.fit_transform(docs)
print(X.toarray())
# Document 1: [0, 1, 1, 1, 0, 2, 0] ← 'pizza' appears twice
# Document 2: [0, 1, 1, 1, 2, 0, 0] ← 'pasta' appears twice
# Document 3: [1, 0, 1, 0, 1, 1, 1] ← each word appears once
```

BoW vs TF-IDF: BoW gives equal weight to all words. TF-IDF downweights common words. TF-IDF is almost always better for ML.

Typical Text Preprocessing Pipeline

```
import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

def clean_text(text):
    # 1. Lowercase
    text = text.lower()
    # 2. Remove non-alphabetic characters
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    # 3. Tokenize
    tokens = word_tokenize(text)
    # 4. Remove stop words
    tokens = [t for t in tokens if t not in stopwords.words("english")]
    # 5. Stem
    stemmer = PorterStemmer()
    tokens = [stemmer.stem(t) for t in tokens]
    # 6. Join back
    return " ".join(tokens)

corpus_clean = [clean_text(doc) for doc in corpus]
```

8. Data Visualization I

Why Visualize?

Raw numbers hide patterns. A histogram of Titanic fares immediately shows you that most tickets were cheap (under 50) with a few luxury tickets at the top. A table of 891 fare values would never make this obvious.

matplotlib vs seaborn

Library	Philosophy	Syntax
matplotlib	Low-level, full control	<code>plt.plot()</code> , <code>plt.hist()</code> , <code>plt.title()</code>
seaborn	High-level, built on matplotlib, works directly with DataFrames	<code>sns.histplot()</code> , <code>sns.boxplot()</code> , <code>sns.countplot()</code>

You'll almost always use seaborn for quick analysis and matplotlib for customizing the final look:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(data=df, x="Fare", bins=30)
plt.title("Distribution of Ticket Fares on Titanic") # matplotlib customization
plt.xlabel("Fare ($)")
plt.ylabel("Count")
plt.show()
```

Titanic Dataset — Quick Reference

- 891 rows, 12 columns
- **Survived:** 0 = died (549), 1 = survived (342)
- **Pclass:** 1st (upper), 2nd (middle), 3rd (lower) class
- **Sex:** male (577), female (314)
- **Age:** 0.42 to 80, 177 missing
- **Fare:** 0 to 512, mean = 32, heavily right-skewed
- **Embarked:** C = Cherbourg, Q = Queenstown, S = Southampton

Histogram — Distribution of One Numeric Variable

```
sns.histplot(df["Fare"], bins=30, kde=True)
```

What to look for:

- **Shape:** Is it symmetric, skewed, bimodal (two peaks)?
- **Peaks:** Where does most data concentrate?
- **Spread:** What's the range?
- **Gaps:** Are there values with no data?

Titanic Fare histogram observation:

- Highly right-skewed (long tail to the right)
- Most tickets cost between 0 and 50 dollars
- A small peak around 50-100
- A few extremely expensive tickets (>500)
- This makes sense: mostly 3rd class (cheap) passengers, few 1st class (expensive)

Bins — How Many?

- Too few bins (5): Lose all detail, just see a block
- Too many bins (100): Noisy, hard to see the pattern
- Rule of thumb: Start with 30 and adjust

```
sns.histplot(df["Age"], bins=10) # Chunkier view
sns.histplot(df["Age"], bins=50) # More detailed
```

KDE — Smooth Version of Histogram

```
sns.kdeplot(df["Fare"], fill=True) # Shaded area under curve
sns.histplot(df["Fare"], kde=True) # Histogram with KDE overlay
```

KDE vs Histogram:

- Histogram: bars, depends on bin width
- KDE: smooth curve, independent of bins
- KDE can show you subtle patterns that bin choice might hide

Countplot — Bar Chart for Categories

```
sns.countplot(data=df, x="Survived")
# Survived: 0 = 549 (died), 1 = 342 (survived)
# More people died than survived – 62% vs 38%

sns.countplot(data=df, x="Sex", hue="Survived")
# More males than females overall, but more females survived
```

Observations from Countplots (Titanic)

- "Women and children first" policy is visible — higher proportion of women survived
- Lower class passengers had lower survival rates
- These are patterns you can see BEFORE any fancy ML

Missing Value Handling — Why Median for Age?

```
df["Age"].fillna(df["Age"].median(), inplace=True)
```

Why median and not mean? Age is slightly skewed. Mean would be pulled by extremes (older passengers). Median (28) is more representative of a "typical" passenger than mean (29.7). For skewed data, median is almost always the better choice.

What about mode for categorical columns?

```
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
# For Embarked, mode is 'S' (Southampton)
```

Categorical vs Numerical Visualization Guide

Variable Type	Best Plot	What it shows
Numerical	Histogram	Shape, spread, skewness
Numerical	Boxplot	Quartiles, outliers
Numerical	KDE Plot	Smooth distribution
Categorical	Countplot	Frequency per category
Categorical vs Numerical	Boxplot (x=cat, y=num)	Compare distributions across groups
Categorical vs Numerical	Barplot (x=cat, y=num_mean)	Mean value per category

9. Data Visualization II

Boxplot — Complete Breakdown

A boxplot gives you a 5-number summary visually. It's the best way to compare distributions across groups.

Drawing a boxplot step by step:

1. Sort the data
2. Find median, Q1, Q3
3. Draw a box from Q1 to Q3
4. Draw a line at the median
5. Calculate $IQR = Q3 - Q1$
6. Lower whisker = $\max(\text{min value}, Q1 - 1.5 \times IQR)$
7. Upper whisker = $\min(\text{max value}, Q3 + 1.5 \times IQR)$
8. Anything beyond whiskers = individual points (outliers)

Code — Titanic Age by Sex and Survival

```
sns.boxplot(data=titanic, x="Sex", y="Age", hue="Survived")
```

This creates 4 boxplots:

- Male / Survived = 0
- Male / Survived = 1
- Female / Survived = 0
- Female / Survived = 1

What to Observe in These Boxplots

Observations you should be able to write:

1. **Median ages:** Non-surviving males have a slightly higher median age than surviving males, suggesting younger men were more likely to survive.
2. **Children (low age) across all groups** tend to have higher survival — the "women and children first" policy is visible.
3. **Female survivors** have a wide age range (infants to elderly), confirming that women of ALL ages had priority.
4. **Outliers:** Some very old passengers in both groups (80+ years).
5. **IQR comparison:** The spread of ages is similar across groups, but the shift in medians tells the story.

The **hue** Parameter

hue adds a third dimension. Without hue:

```
sns.boxplot(data=titanic, x="Sex", y="Age")
# 2 boxes: Male, Female
```

With hue:

```
sns.boxplot(data=titanic, x="Sex", y="Age", hue="Survived")
# 4 boxes: Male-Died, Male-Survived, Female-Died, Female-Survived
```

Hue makes comparisons within groups possible. Did younger males have better survival? Hue answers this.

Histogram vs Boxplot — When to Use Which

Situation	Histogram	Boxplot
Show the shape (bimodal, normal, skewed)	✓ Excellent	✗ Hides shape
Compare many groups (10+ categories)	✗ Too cluttered	✓ Compact
Detect outliers	✗ Hard to see	✓ Shows clearly
Quick overview of a single variable	✓ Easy to read	✓ Compact
Identify peaks and gaps	✓ Shows clearly	✗ Hides them
Show exact median and quartiles	✗ Approximate	✓ Exact numbers

Rule of thumb: Use histogram for EXPLORING one variable. Use boxplot for COMPARING across groups.

10. Data Visualization III

Iris Dataset — Feature Understanding

The Iris dataset is PERFECT for visualization because:

- Features have different distributions per species
- Some features separate species well (petal length), others don't (sepal width)
- You can clearly see which features are most useful for classification

Numeric vs Nominal vs Ordinal — Applied to Iris

Data Type	Iris Example	Meaning
Numeric (Continuous)	Sepal Length (4.3-7.9), Petal Width (0.1-2.5)	Can take any value in a range
Numeric (Discrete)	Count of petals (always 0)	Integer values only
Nominal	Species (setosa, versicolor, virginica)	Names, no order
Ordinal	Not in Iris, but e.g., Size (Small < Medium < Large)	Ordered categories

Histogram per Feature — What to Look For

```
df.hist(figsize=(10, 8), bins=20)
```

This gives you 4 histograms (one per feature) without separating by species.

Observations:

1. **Petal Length:** Bimodal (two peaks) — one peak ~1.5 (setosa), another peak ~4.5 (versicolor/virginica). This suggests petal length is a good feature for separating setosa from the others.
2. **Petal Width:** Also bimodal — similar story as petal length.
3. **Sepal Length:** Roughly normal (single peak), slight overlap between species.
4. **Sepal Width:** Roughly normal, but the three species overlap significantly — this is the LEAST useful feature for classification.

Better: Histogram with Hue

```
sns.histplot(data=df, x="PetalLength", hue="Species", kde=True, alpha=0.5)
```

Now you see:

- **Setosa** (blue): Tight peak at ~1.5 — very distinct from others
- **Versicolor** (orange): Peak at ~4.5
- **Virginica** (green): Peak at ~5.5
- Versicolor and virginica overlap around 5.0 — this is where classification gets tricky

Boxplot per Feature — Species Comparison

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
sns.boxplot(data=df, x="Species", y="SepalLength", ax=axes[0,0])
sns.boxplot(data=df, x="Species", y="SepalWidth", ax=axes[0,1])
sns.boxplot(data=df, x="Species", y="PetalLength", ax=axes[1,0])
sns.boxplot(data=df, x="Species", y="PetalWidth", ax=axes[1,1])
```

What each boxplot tells you:

Feature	Setosa	Versicolor	Virginica
Petal Length	1.0–1.9 (tight)	3.0–5.1	4.5–6.9
Petal Width	0.1–0.6 (tight)	1.0–1.8	1.4–2.5
Sepal Length	4.3–5.8	4.9–7.0	4.9–7.9
Sepal Width	2.3–4.4	2.0–3.4	2.2–3.8

Key insight: Petal measurements separate species much better than sepal measurements. This is why classifiers trained on Iris tend to rely heavily on petal features.

Outlier Detection from Boxplots

- **Setosa:** Very tight, almost no outliers. Setosa is a homogeneous group.
- **Versicolor:** A few outliers in sepal width — some unusually narrow sepals.
- **Virginica:** Occasional outliers in various features.

Writing Observations — What Examiners Want

A good observation should be specific and data-driven:

✓ **Good:** "Setosa has petal lengths between 1.0 and 1.9 cm with a median of 1.5 cm, which is clearly separated from versicolor (3.0–5.1 cm) and virginica (4.5–6.9 cm). This makes petal length the most useful feature for distinguishing setosa from the other two species."

✗ **Bad:** "The boxplot shows some differences between species."

Structure for writing observations:

1. State what you see (specific numbers)
2. Compare groups
3. State the implication (what does this mean for the analysis?)

12. WeatherAUS SVM

Weather Dataset Overview

- ~145,000 daily weather records from ~50 Australian stations

- **Target:** `RainTomorrow` — will it rain tomorrow? (Yes/No)
- **Features:** Temperature, humidity, pressure, wind speed, wind direction, rainfall today, cloud cover, etc.
- **Challenge:** Mixed data types (numeric and categorical), missing values, class imbalance (~78% No, ~22% Yes)

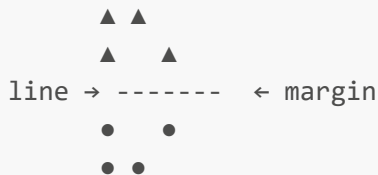
Classification Task

Predict a binary outcome: `RainTomorrow` = Yes or No. This is NOT a regression problem even though we're predicting "how much" implicitly — we only care about whether it rains, not how much.

SVM — Support Vector Machine

SVM finds a hyperplane that separates classes with the maximum possible margin.

Simple 2D example: Imagine data points with two features (x_1, x_2). Blue circles = class A, red triangles = class B. SVM draws a line between them. But not just ANY line — the line that has the maximum distance to the closest points on both sides.



Key SVM Concepts — In Detail

Hyperplane: The decision boundary. In 2D, it's a line. In 3D, it's a plane. In higher dimensions (your actual data), it's a hyperplane.

Support Vectors: The data points that are CLOSEST to the hyperplane. These are the "support" vectors — they define where the hyperplane goes. Other points farther away don't matter at all.

Margin: The distance between the hyperplane and the nearest support vectors. SVM maximizes this margin.

Why maximize the margin?

- Small margin = boundary is tight around the data. A slightly different test point might be misclassified.
- Large margin = boundary is comfortably far from all points. More robust to new data.

The beauty of SVM: Only support vectors matter. Move a non-support-vector point and the boundary doesn't change. This makes SVM computationally efficient.

Kernel Trick — Handling Non-Linear Data

What if the data isn't linearly separable (no straight line can separate the classes)?

Idea: Map the data to a HIGHER dimension where it BECOMES linearly separable.

Example — 1D to 2D:

```

1D:      0  1  -1  2  -2
Classes: A  A  B  A  B
          ↑ no line can separate these!

Add a second dimension:  $x_2 = x_1^2$ 
2D:      (0,0) (1,1) (-1,1) (2,4) (-2,4)
Classes:  A    A    B    A    B
          ↑ now a line can separate them!

```

This is called the **kernel trick** — SVM computes the dot product in the higher-dimensional space WITHOUT actually transforming the data. It's a mathematical shortcut.

```

from sklearn.svm import SVC

# Linear kernel – straight line boundary
svm_linear = SVC(kernel="linear")

# Polynomial kernel – curved boundary
svm_poly = SVC(kernel="poly", degree=3)

# RBF kernel – complex boundaries (default, most flexible)
svm_rbf = SVC(kernel="rbf", gamma="scale")

```

Kernel	What the boundary looks like	When to use
Linear	Straight line/plane	Data is linearly separable
Polynomial	Curved, wavy	Moderate non-linearity
RBF	Any shape (circles, blobs, squiggles)	Complex patterns, default choice

Label Encoding — Weather Data Categorical Columns

WeatherAUS has several categorical columns:

- Location (49 weather stations)
- WindGustDir (16 wind directions: N, NNE, NE, ...)
- WindDir9am, WindDir3pm
- RainToday (Yes/No)

```

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

categorical_cols = ["Location", "WindGustDir", "WindDir9am", "WindDir3pm",
"RainToday"]
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

```

Why not one-hot for Location (49 categories)? 49 columns would make the dataset too wide. Label encoding is acceptable here.

Feature Scaling — Why SVM Absolutely Needs It

SVM computes distances between points to find the hyperplane. If one feature (Pressure: 1000-1040 hPa) has much larger values than another (Humidity: 0-100), the distance calculation is dominated by the larger feature.

Without scaling: Pressure difference of 1 unit counts as much as humidity difference of 100 units. **With scaling:** Both features contribute equally.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# Scale features (NOT the target!)
X_scaled = scaler.fit_transform(X)
```

Data Leakage — The #1 Mistake in WeatherAUS

Data leakage: When information from the FUTURE (or from outside your training set) leaks into your training features, making your model artificially accurate.

RISK_MM — The classic trap:

- **RISK_MM** stands for "Risk in millimeters" — it's the amount of rainfall recorded TODAY
- **RainTomorrow** is whether it will rain TOMORROW
- If today has high rainfall (**RISK_MM** > 0), it's very likely to rain tomorrow too
- **Problem:** **RISK_MM** is measured at the END of today, same time as other features. But in a real prediction scenario, you wouldn't know today's total rainfall until the day is over. More importantly, it's essentially the same phenomenon as the target.

Result: Models with **RISK_MM** achieve ~85% accuracy. Models without it drop to ~78-80%. The high accuracy was fake.

What to do:

```
df.drop("RISK_MM", axis=1, inplace=True) # Always remove before training
```

Other Data Leakage Examples

Scaling before splitting:

```
# WRONG — test data influences scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) # Fits on ALL data including test
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)
```

```
# RIGHT
X_train, X_test, y_train, y_test = train_test_split(X, y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # Uses train's mean/std
```

RainToday vs RainTomorrow vs RISK_MM

Column	What it means	Can you use it?
RainToday	Did it rain today? (Yes/No)	✓ Yes — it's a known fact at prediction time
RainTomorrow	Will it rain tomorrow? (Yes/No)	✗ No — this is the TARGET (what you're predicting)
RISK_MM	How much rain fell today (in mm)	✗ No — causes data leakage

SVM vs Other Classifiers for Weather Data

Classifier	Why it might struggle with weather data
Logistic Regression	Assumes linear decision boundary — weather patterns are non-linear
Naive Bayes	Assumes feature independence — temperature and humidity are correlated
SVM with RBF kernel	Can capture complex non-linear patterns — best choice here

Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Interpreting results:

- Accuracy ~80% is decent for weather prediction
- Check if the model is just predicting "No rain" for everyone (78% accuracy, but 0% recall for rain!)
- Confusion matrix reveals: how many rainy days did we actually catch?
- Classification report shows: precision/recall per class

Quick Reference — Must-Know for Viva

Data Cleaning

Concept	Key Point
Missing values	Check with <code>isnull().sum()</code> . Handle with <code>fillna()</code> (mean/median/mode) or <code>dropna()</code> .
Outliers	IQR method: $Q1 - 1.5 \times IQR$ to $Q3 + 1.5 \times IQR$. Remove or cap.
Label Encoding	Maps categories to 0,1,2... Good for ordinal. Risky for nominal.
One-Hot Encoding	Creates binary columns. Good for nominal with few categories.
Min-Max Scaling	Scales to [0,1]. Sensitive to outliers.
Standardization	Fixes mean=0, std=1. Robust to outliers.

Statistics (Descriptive)

Concept	Formula	Sensitive to outliers?
Mean	$\Sigma x / n$	Yes
Median	Middle value	No
Std Dev	$\sqrt{(\Sigma(x-\mu)^2/n)}$	Yes
IQR	$Q3 - Q1$	No

ML Models

Model	Type	What it Does	Key Parameter
Linear Regression	Regression	Fits a line to minimize squared error	—
Logistic Regression	Binary Classification	Sigmoid $\rightarrow$ probability $\rightarrow$ threshold	C (inverse regularization)
Naive Bayes	Classification (multiclass)	Bayes theorem with independence assumption	—
SVM	Classification	Maximize margin between classes	kernel, C, gamma

Evaluation Metrics

Metric	Formula	When to Care
MAE	$\text{mean}(y - \hat{y})$	Regression — interpretable error
MSE	$\text{mean}((y - \hat{y})^2)$	Regression — penalizes large errors
RMSE	$\sqrt{\text{MSE}}$	Regression — same unit as target
R^2	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Regression — variance explained
Accuracy	$(\text{TP} + \text{TN})/(\text{total})$	Balanced classes

Metric	Formula	When to Care
Precision	$TP/(TP+FP)$	When FP is costly
Recall	$TP/(TP+FN)$	When FN is costly
F1	$2PR/(P+R)$	Balance of P and R

Text Analytics Pipeline

1. Lowercase → 2. Remove punctuation/numbers (regex) → 3. Tokenize → 4. Remove stop words → 5. Stem/Lemmatize → 6. Convert to TF-IDF

Datasets Summary

Dataset	Size	Task	Target
Iris	150×5	Classify species	Species (3 classes)
Titanic	891×12	Predict survival	Survived (0/1)
Boston Housing	506×14	Predict price	MEDV (continuous)
Social Network Ads	400×3	Predict purchase	Purchased (0/1)
WeatherAUS	$145K \times 23$	Predict rain tomorrow	RainTomorrow (Yes/No)

Libraries Reference — What Each Library Does

Covers all Python libraries used across practicals — import style, key methods, extra methods for viva.

pandas (`import pandas as pd`)

Purpose: Data manipulation and analysis. The backbone of all data wrangling practicals.

Core Data Structures:

Structure	Description	Creation
<code>DataFrame</code>	2D labeled table (rows & columns)	<code>pd.DataFrame(dict), pd.read_csv()</code>
<code>Series</code>	1D labeled array (single column)	<code>df["col"], pd.Series(list)</code>

I/O — Reading & Writing Data:

```
pd.read_csv("file.csv")           # CSV file
pd.read_excel("file.xlsx")        # Excel file
pd.read_json("file.json")         # JSON file
pd.read_sql("SELECT * FROM t", con) # SQL database
df.to_csv("output.csv", index=False) # Write CSV (index=False to avoid extra
```

```
column)
df.to_excel("output.xlsx", index=False)
```

Viva question: What does `index=False` do? It prevents pandas from writing row numbers as a column.

Inspecting Data:

Method	What it returns
<code>df.head(n)</code>	First n rows (default 5)
<code>df.tail(n)</code>	Last n rows
<code>df.sample(n)</code>	Random n rows
<code>df.info()</code>	Column names, non-null counts, dtypes
<code>df.describe()</code>	Summary statistics for numeric cols
<code>df.describe(include="object")</code>	Summary for categorical cols (count, unique, top, freq)
<code>df.shape</code>	(rows, columns) tuple
<code>df.columns</code>	Column names
<code>df.dtypes</code>	Data type of each column
<code>df.values</code>	Underlying numpy array
<code>df.index</code>	Row index

Selecting Data:

```
df["col"]           # Single column → Series
df[["col1", "col2"]] # Multiple columns → DataFrame
df.loc[rows, cols]  # Select by LABEL
df.iloc[rows, cols] # Select by INTEGER position
df.iloc[0:5, 0:3]   # First 5 rows, first 3 columns
df[df["Age"] > 30]   # Boolean/filtering
df.query("Age > 30") # Query-style filtering
```

loc vs iloc — Viva classic:

- `df.loc[0:5]` includes row 5 (label-based, inclusive)
- `df.iloc[0:5]` excludes row 5 (integer-based, exclusive — like Python slicing)

Handling Missing Values:

Method	What it does
<code>df.isnull()</code>	Returns True/False per cell
<code>df.isnull().sum()</code>	Count of NaN per column

Method	What it does
<code>df.isnull().sum().sum()</code>	Total NaN in entire df
<code>df.notnull()</code>	Opposite of isnull
<code>df.dropna()</code>	Drop rows with any NaN
<code>df.dropna(subset=["col"])</code>	Drop rows where specific col is NaN
<code>df.dropna(axis=1)</code>	Drop columns with any NaN
<code>df.fillna(value)</code>	Replace NaN with a value
<code>df.fillna(method="ffill")</code>	Forward fill (carry previous value forward)
<code>df.fillna(method="bfill")</code>	Backward fill
<code>df.interpolate()</code>	Linear interpolation between values

Extra method for viva — `interpolate()`: Fills NaN by drawing a straight line between surrounding values. Useful for time series data.

```
# Example: [10, NaN, NaN, 25] → [10, 15, 20, 25]
df["col"].interpolate()
```

Datatype Operations:

```
df["col"].astype("int")           # Convert type
pd.to_datetime(df["date_col"])    # Convert to datetime
pd.to_numeric(df["col"], errors="coerce") # Convert, invalid → NaN
```

`errors="coerce"` — Viva note: When converting strings to numbers, if a value can't be converted (e.g., "N/A"), `errors="coerce"` turns it into NaN instead of crashing.

Duplicates:

```
df.duplicated()                  # Boolean Series
df.duplicated().sum()            # Count duplicates
df.drop_duplicates()             # Remove duplicates
df.drop_duplicates(subset=["col"], keep="last") # Keep last occurrence
```

Sorting & Ranking:

```
df.sort_values("col", ascending=False) # Sort by column
df.sort_index()                        # Sort by index
df["rank"] = df["col"].rank()          # Rank values (1 = smallest)
```


Applying Functions:

```
df["col"].apply(lambda x: x**2)          # Apply function to a Series
df.applymap(lambda x: x*2)                # Apply to every element in df
(element-wise)
df["col"].map({"Male": 0, "Female": 1})   # Map values using dict
```

apply() vs map(): `map()` is for Series only and replaces values. `apply()` can work on Series or DataFrame and runs a function.

Grouping & Aggregation:

```
df.groupby("cat_col")["num_col"].mean()    # Single aggregation
df.groupby("cat_col").agg({"col1": "mean", "col2": "sum"}) # Multiple
aggregations
df.groupby("cat_col").agg(["mean", "std", "min", "max"])   # Multiple functions
df.pivot_table(values="val", index="row", columns="col", aggfunc="mean") # Pivot
table
```

Merging/Joining DataFrames:

```
pd.merge(df1, df2, on="key_col")          # SQL-style join
pd.concat([df1, df2], axis=0)              # Stack rows (append)
pd.concat([df1, df2], axis=1)              # Stack columns (side by side)
```

merge() vs concat(): `merge()` joins on a key (like SQL JOIN). `concat()` just stacks (like adding rows or columns).

Sampling & Random:

```
df.sample(n=100)                          # Random n rows
df.sample(frac=0.1)                       # Random 10% of rows
df.sample(frac=1).reset_index(drop=True)   # Shuffle entire dataset
```

numpy (`import numpy as np`)

Purpose: Numerical computing — arrays, math operations, random numbers. Used indirectly (pandas is built on numpy) and directly for transformations.

Core Object — ndarray:

```
arr = np.array([1, 2, 3, 4, 5])           # 1D array
```

```
arr2d = np.array([[1,2], [3,4]])      # 2D array
```

Creating Arrays:

```
np.zeros((3, 4))      # 3x4 array of zeros
np.ones((2, 5))       # 2x5 array of ones
np.arange(0, 10, 2)   # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)  # [0, 0.25, 0.5, 0.75, 1]
np.random.randn(100)  # 100 random numbers from normal distribution
np.random.randint(0, 10, size=20) # 20 random integers 0-9
```

Math Operations (used in your practicals):

```
np.log(x)              # Natural log – for data transformation
np.log1p(x)            # log(1 + x) – handles zeros safely
np.sqrt(x)             # Square root – for data transformation
np.exp(x)              # e^x – inverse of log
np.abs(x)              # Absolute value – used for MAE
np.square(x)           # x^2 – used for MSE
np.mean(x)             # Mean
np.median(x)           # Median
np.std(x)              # Standard deviation
np.var(x)              # Variance
np.min(x), np.max(x)   # Min / Max
np.sum(x)              # Sum
np.corrcoef(a, b)      # Correlation coefficient
```

Why `np.log1p` instead of `np.log`? `log(0)` is undefined ($-\infty$). If your data has zeros, use `log1p(x) = log(1+x)`. Also relevant: `np.log10(x)` for base-10 log, `np.log2(x)` for base-2.

Random — For Reproducibility:

```
np.random.seed(42)      # Fix random seed – get same "random" numbers every time
np.random.randn(100)    # Now deterministic
```

Viva question: Why `random.seed`? Without a seed, every run gives different random numbers. With a seed, results are reproducible — essential for debugging and experiments.

Array Operations:

```
arr + 5                # Broadcast: add 5 to every element
arr1 + arr2            # Element-wise addition
arr.mean()             # Mean of array
arr.std()              # Std dev
```

```
arr.reshape(2, 3)      # Reshape to 2 rows, 3 cols
arr.flatten()          # Flatten to 1D
```

matplotlib (`import matplotlib.pyplot as plt`)

Purpose: Low-level plotting. Every seaborn plot ultimately calls matplotlib under the hood.

Basic Plot Types:

```
plt.plot(x, y)          # Line plot
plt.scatter(x, y)       # Scatter plot
plt.bar(x, height)      # Bar chart
plt.hist(data, bins=30) # Histogram
plt.boxplot(data)       # Boxplot
plt.pie(sizes, labels=labels) # Pie chart
```

Customization — What examiners ask about:

```
plt.figure(figsize=(10, 6)) # Figure size (width, height in inches)
plt.title("Title")          # Title
plt.xlabel("X-axis label")  # X-axis label
plt.ylabel("Y-axis label")  # Y-axis label
plt.xlim(0, 100)           # X-axis range
plt.ylim(0, 1)             # Y-axis range
plt.xticks(rotation=45)     # Rotate x labels
plt.grid(True)              # Show grid
plt.legend()                # Show legend (must have label= in plot)
plt.tight_layout()          # Auto-adjust spacing
plt.savefig("plot.png", dpi=300) # Save to file
plt.show()                  # Display plot
plt.clf()                   # Clear current figure
plt.close()                 # Close figure window
```

Subplots — Multiple plots in one figure:

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
# axes is a 2x2 array of Axes objects
axes[0,0].hist(df["col1"])
axes[0,1].scatter(df["x"], df["y"])
axes[1,0].boxplot(df["col3"])
plt.tight_layout()
plt.show()
```

Viva question: Why `plt.tight_layout()`? Without it, subplot labels and titles often overlap.
`tight_layout()` auto-adjusts padding.

Key concepts:

- **Figure:** The entire window/page
- **Axes:** One individual plot (NOT the plural of axis — it's a plotting area with its own axes)
- **Axis:** The x or y axis line with ticks and labels

```
fig = plt.figure()           # Empty figure
ax = fig.add_subplot(1,1,1)  # Add one axes
ax.plot(x, y)                # Plot on that axes
```

Common patterns used in practicals:

```
# Histogram
plt.hist(df["Age"], bins=20, edgecolor="black")
plt.title("Age Distribution")
plt.show()

# Scatter with customization
plt.scatter(df["Age"], df["Fare"], alpha=0.5, c=df["Survived"], cmap="coolwarm")
plt.colorbar(label="Survived")
plt.show()
```

seaborn (`import seaborn as sns`)

Purpose: High-level statistical visualization. Built on matplotlib, works directly with DataFrames, has beautiful defaults.

Setup:

```
sns.set_theme()              # Apply seaborn defaults
sns.set_style("whitegrid")    # Set style: darkgrid, whitegrid, dark, white,
ticks                         # Color palette
sns.set_palette("husl")
```

Plot Types Used in Practical:

```
# Distribution plots
sns.histplot(data=df, x="col", bins=30, kde=True)      # Histogram + KDE
sns.kdeplot(data=df, x="col", fill=True)              # KDE only
sns.displot(data=df, x="col", kind="hist")            # Distribution
# (figure-level)
sns.rugplot(data=df, x="col")                          # Rug marks (each
# data point as a tick)

# Categorical plots
```

```

sns.countplot(data=df, x="cat_col") # Count per category
sns.countplot(data=df, x="cat_col", hue="another_cat") # Grouped counts
sns.boxplot(data=df, x="cat_col", y="num_col") # Boxplot
sns.boxplot(data=df, x="cat_col", y="num_col", hue="cat2") # Grouped boxplot
sns.violinplot(data=df, x="cat_col", y="num_col") # Violin = boxplot +
KDE
sns.barplot(data=df, x="cat_col", y="num_col") # Bar = mean per
group with CI
sns.pointplot(data=df, x="cat_col", y="num_col") # Point estimates

# Relationship plots
sns.scatterplot(data=df, x="col1", y="col2") # Scatter
sns.scatterplot(data=df, x="col1", y="col2", hue="cat") # Colored by category
sns.lineplot(data=df, x="col1", y="col2") # Line plot
sns.regplot(data=df, x="col1", y="col2") # Scatter +
regression line
sns.lmplot(data=df, x="col1", y="col2", hue="cat") # Regression per
group

# Heatmap (correlation matrix)
sns.heatmap(df.corr(), annot=True, cmap="coolwarm") # Correlation heatmap

# Pairplot (scatter matrix – all pairs of features)
sns.pairplot(data=df, hue="Species") # Iris classic

```

figure-level vs axes-level — Viva question:

- **Axes-level** (histplot, boxplot, scatterplot, etc.): Returns a matplotlib Axes object. Can be placed in subplots with `ax=`.
- **Figure-level** (displot, relplot, catplot, lmplot, pairplot): Creates an entire figure. Cannot be placed in subplots.

```

# Axes-level: place in subplot
fig, ax = plt.subplots()
sns.histplot(data=df, x="col", ax=ax)

# Figure-level: creates its own figure
sns.displot(data=df, x="col", col="Species", row="Sex") # Facet grid!

```

hue parameter — Adds dimension via color:

```

sns.boxplot(data=df, x="Sex", y="Age", hue="Survived")
# Four boxes: Male-Died, Male-Survived, Female-Died, Female-Survived

```

col and row — Faceting (create multiple subplots by category):

```
sns.displot(data=df, x="Age", col="Sex", row="Survived")
# 2x2 grid: Male-Died, Male-Survived, Female-Died, Female-Survived
```

Palettes for viva:

```
sns.color_palette()           # Default
sns.color_palette("husl")     # HUSL (perceptually uniform)
sns.color_palette("coolwarm") # Diverging
sns.color_palette("viridis")  # Viridis (colorblind-friendly)
sns.color_palette("Set1")     # Qualitative (for categories)
```

Common tricks:

```
sns.set_theme(style="whitegrid", font_scale=1.2) # Bigger fonts
sns.despine()                                  # Remove top/right spines
```

scikit-learn (`import sklearn`)

Purpose: Machine learning library. Every ML practical (4, 5, 6, 12) uses it.

Submodules used in your practicals:

Submodule	Import As	Used In
<code>sklearn.preprocessing</code>	—	Data Wrangling, Logistic Regression, SVM
<code>sklearn.model_selection</code>	—	All ML practicals
<code>sklearn.linear_model</code>	—	Linear Regression, Logistic Regression
<code>sklearn.naive_bayes</code>	—	Naive Bayes
<code>sklearn.svm</code>	—	SVM
<code>sklearn.metrics</code>	—	All ML practicals
<code>sklearn.feature_extraction.text</code>	—	Text Analytics

A) `sklearn.preprocessing`

Class/Function	What it does	Code
<code>StandardScaler</code>	Z-score scaling (mean=0, std=1)	<code>scaler = StandardScaler()</code> then <code>scaler.fit_transform(X)</code>
<code>MinMaxScaler</code>	Min-max scaling ([0,1])	Same pattern

Class/Function	What it does	Code
<code>LabelEncoder</code>	Encode categories as 0,1,2...	<code>le.fit_transform(df["col"])</code>
<code>OneHotEncoder</code>	One-hot encode categories	<code>ohe.fit_transform(df[["col"]])</code>
<code>Binarizer</code>	Threshold — values above threshold → 1, else 0	<code>Binarizer(threshold=0.5).transform(X)</code>
<code>Normalizer</code>	Scale rows to unit length (L1/L2 norm)	Different from <code>MinMaxScaler</code> — used for text data

Extra for viva:

```
from sklearn.preprocessing import RobustScaler, PolynomialFeatures

# RobustScaler – uses median and IQR. Even more robust to outliers than
# StandardScaler.
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)

# PolynomialFeatures – creates interaction terms ( $x_1^2$ ,  $x_1 \times x_2$ , etc.)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
# Used when you suspect non-linear relationships in linear regression
```

Viva question: `StandardScaler` vs `RobustScaler`?

- `StandardScaler` uses mean/std (sensitive to outliers).
- `RobustScaler` uses median/IQR (ignores outliers).
- If data has outliers, `RobustScaler` is better.

Viva question: `fit`, `transform`, `fit_transform` — what's the difference?

- `fit()`: Learns parameters (mean/std for `StandardScaler`) from the data. Does NOT change the data.
- `transform()`: Applies the transformation using learned parameters.
- `fit_transform()`: fit + transform in one step. Use on TRAINING data only.
- For TEST data: only `transform()` (using parameters learned from training).

B) `sklearn.model_selection`

Function	What it does	Code
<code>train_test_split</code>	Split data into train/test	<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>
<code>cross_val_score</code>	K-fold cross-validation	<code>cross_val_score(model, X, y, cv=5)</code>
<code>KFold</code>	K-fold split indices	<code>KFold(n_splits=5)</code>

Function	What it does	Code
<code>StratifiedKFold</code>	K-fold preserving class ratios	For imbalanced datasets

Parameters of `train_test_split`:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # 20% for testing
    train_size=0.8,         # or 80% for training (redundant if test_size given)
    random_state=42,        # For reproducibility
    stratify=y,             # Preserve class distribution in both splits
    shuffle=True            # Shuffle before splitting (default)
)
```

Viva question: What is `stratify`? When classes are imbalanced (e.g., 90% No, 10% Yes), `stratify=y` ensures both training and test sets have the same 90-10 split. Without it, one split might accidentally get all the "Yes" samples.

Cross-validation — Viva concept:

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print(f"Accuracy: {scores.mean():.3f} ± {scores.std():.3f}")
```

Instead of one train-test split, do 5 splits. Each time, 4 folds train, 1 fold tests. Average the results. More reliable estimate of model performance.

C) `sklearn.linear_model`

Model	Code	Use Case
<code>LinearRegression</code>	<code>LinearRegression()</code>	Regression (continuous target)
<code>LogisticRegression</code>	<code>LogisticRegression()</code>	Binary classification

LinearRegression:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)

# Important attributes:
print(model.coef_)           # Coefficients (slope) for each feature
```



```
print(model.intercept_)    # Intercept (bias term)
y_pred = model.predict(X_test)
```

LogisticRegression:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train_scaled, y_train)

print(model.coef_)        # Coefficients (log-odds)
print(model.intercept_)   # Intercept
print(model.predict_proba(X_test)) # Probabilities [P(0), P(1)]
y_pred = model.predict(X_test)
```

Viva question: `predict()` vs `predict_proba()`?

- `predict()` returns class labels (0 or 1).
- `predict_proba()` returns probability scores `[[P(class=0), P(class=1)]]`.
- Default threshold: if $P(1) \geq 0.5$, predict 1.

Parameters for LogisticRegression:

```
LogisticRegression(
    penalty="l2",          # Regularization type (l1, l2, elasticnet)
    C=1.0,                 # Inverse regularization strength (smaller =
stronger)
    solver="lbfgs",        # Optimization algorithm
    max_iter=100           # Max iterations
)
```

Viva question: What's C in LogisticRegression? C = inverse of regularization strength. Smaller C = more regularization (simpler model, prevents overfitting). Larger C = less regularization (model fits training data more closely).

What happens if `max_iter` is too low? The model doesn't converge — throws a "ConvergenceWarning: lbfgs failed to converge". Increase `max_iter` or scale features better.

D) `sklearn.naive_bayes`

Model	Code	Use Case
GaussianNB	GaussianNB()	Continuous features (normal distribution)
MultinomialNB	MultinomialNB()	Discrete counts (text classification)
BernoulliNB	BernoulliNB()	Binary features (present/absent)

```

from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)

# Attributes:
print(model.class_prior_)      # P(Class) – learned priors
print(model.theta_)            # Mean of each feature per class
print(model.sigma_)            # Variance of each feature per class

```

Viva question: Which Naive Bayes for which data?

- **Gaussian:** Petal lengths, temperatures, salaries (continuous, roughly normal)
- **Multinomial:** Word counts in text (e.g., "pizza" appears 3 times)
- **Bernoulli:** Presence/absence (e.g., does "pizza" appear in doc? 0/1)

Viva question: The "naive" assumption? Features are independent given the class. In Iris, petal length and petal width are correlated — the assumption is violated but Naive Bayes still works.

E) `sklearn.svm`

Classifier	Code	Use Case
SVC	<code>SVC(kernel="rbf")</code>	Classification
LinearSVC	<code>LinearSVC()</code>	Faster linear SVM (no kernel trick)
NuSVC	<code>NuSVC()</code>	SVM with nu parameter instead of C

```

from sklearn.svm import SVC
model = SVC(kernel="rbf", C=100, gamma="scale")
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test)

```

Parameters for Viva:

Parameter	What it controls	Low value →	High value →
C	Margin hardness	Wider margin, simpler model (underfit)	Narrower margin, complex model (overfit)
gamma (RBF only)	How far one point influences	Large influence radius, smooth boundary	Small influence radius, wiggly boundary
kernel	Type of boundary	Linear = straight, RBF = flexible, Poly = curved	

Gamma — Visual intuition:

- Low gamma: Each training point influences a wide area. Boundary is smooth, general.
- High gamma: Each point only influences its immediate neighborhood. Boundary is complex, may overfit.

LinearSVC vs SVC(kernel="linear"):

- **LinearSVC** is faster for large datasets. Uses liblinear.
- **SVC(kernel="linear")** uses libsvm, slower but supports more kernels.

F) sklearn.metrics

Regression Metrics:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

# Extra:
from sklearn.metrics import mean_absolute_percentage_error
mape = mean_absolute_percentage_error(y_test, y_pred) * 100 # Percentage error
```

Classification Metrics:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

# Basic
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred) # Returns [[TN, FP], [FN, TP]]
print(classification_report(y_test, y_pred))

# Per-class metrics
precision = precision_score(y_test, y_pred, pos_label=1)
recall = recall_score(y_test, y_pred, pos_label=1)
f1 = f1_score(y_test, y_pred, pos_label=1)

# ROC-AUC – model's ability to separate classes (0.5 = random, 1.0 = perfect)
y_proba = model.predict_proba(X_test)[: , 1]
auc = roc_auc_score(y_test, y_proba)
```

Viva question: When is accuracy misleading? When classes are imbalanced (e.g., 95% No, 5% Yes). A model that always predicts "No" gets 95% accuracy but is useless. Use precision/recall/F1 instead.

classification_report output:

	precision	recall	f1-score	support
0	0.85	0.90	0.87	50
1	0.80	0.72	0.76	25
accuracy			0.83	75
macro avg	0.83	0.81	0.81	75
weighted avg	0.83	0.83	0.83	75

- **Support:** Number of actual samples of that class in the test set
- **Macro avg:** Simple average across classes (treat classes equally)
- **Weighted avg:** Average weighted by support (accounts for class imbalance)

Confusion Matrix Order — Tricky for viva:

```
# sklearn confusion_matrix returns:
# [[TN, FP],
#  [FN, TP]]
# NOT [[TP, FN], [FP, TN]]!

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel() # Unpack in correct order
```

G) `sklearn.feature_extraction.text`

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

# Bag of Words
cv = CountVectorizer(max_features=1000, stop_words="english")
X_bow = cv.fit_transform(documents) # Sparse matrix
vocab = cv.get_feature_names_out() # List of words

# TF-IDF
vectorizer = TfidfVectorizer(max_features=1000, stop_words="english",
                             ngram_range=(1, 2)) # unigrams + bigrams
X_tfidf = vectorizer.fit_transform(documents)
```

Parameters:

- `max_features`: Only use top K most frequent words
- `stop_words`: "english" to remove stop words automatically
- `ngram_range`: (1,1) = single words, (1,2) = words + word pairs, (2,2) = only word pairs
- `min_df`: Minimum document frequency (ignore words appearing in fewer than min_df docs)
- `max_df`: Maximum document frequency (ignore words appearing in more than max_df fraction of docs — these are probably stop words)

Extra for viva — `ngram_range`:

```
# Unigrams only: "not", "good"
TfidfVectorizer(ngram_range=(1,1))

# Unigrams + bigrams: "not", "good", "not good"
TfidfVectorizer(ngram_range=(1,2))

# Bigrams only: "not good"
TfidfVectorizer(ngram_range=(2,2))
```

Using bigrams helps capture negation ("not good" vs "good") — important for sentiment analysis.

H) `sklearn.pipeline` (Extra)

```
from sklearn.pipeline import Pipeline

# Chain preprocessing + model into one object
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

# This automatically:
# 1. Scales X_train, fits model
# 2. Uses same scaler to transform X_test, then predicts
# No chance of data leakage!
```

Why use pipelines? They prevent data leakage, reduce code, and ensure the exact same preprocessing is applied to train and test.

`nltk` (`import nltk`)

Purpose: Natural Language Toolkit. All text preprocessing tasks.

Setup — Downloading resources:

```
nltk.download("punkt")           # Tokenizer models
nltk.download("stopwords")       # Stop words list
nltk.download("wordnet")         # WordNet dictionary (for lemmatization)
nltk.download("averaged_perceptron_tagger") # POS tagger model
nltk.download("omw-1.4")        # Open Multilingual WordNet
```

Viva question: Why do you need to `nltk.download()`? NLTK resources are not included with the library (would be too large). You download them once, and they're cached locally.

Tokenization:

```
from nltk.tokenize import word_tokenize, sent_tokenize, RegexpTokenizer

# Word tokenization
tokens = word_tokenize("Hello world!")      # → ["Hello", "world", "!"]

# Sentence tokenization
sents = sent_tokenize("Hi! How are you?")   # → ["Hi!", "How are you?"]

# Regexp tokenizer – custom pattern
tokenizer = RegexpTokenizer(r"\w+")          # Only alphanumeric tokens
tokens = tokenizer.tokenize("Hello, world!") # → ["Hello", "world"]
```

RegexpTokenizer vs word_tokenize: `word_tokenize` is smarter (handles "N.Y.C.", "Dr." as one token). `RegexpTokenizer` is faster and gives you control.

Stop Words:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
# Also available: "french", "german", "spanish", etc.
```

Stemming:

```
from nltk.stem import PorterStemmer, LancasterStemmer, SnowballStemmer

ps = PorterStemmer()          # Most common, gentle
ls = LancasterStemmer()       # More aggressive (shorter stems)
ss = SnowballStemmer("english") # Improved version of Porter

ps.stem("running")            # → "run"
ls.stem("running")            # → "run"
ls.stem("studies")            # → "studi" (aggressive)
```

Stemmer comparison — viva:

- **Porter:** Gentle, well-known, produces readable stems
- **Lancaster:** Aggressive, shorter stems, may produce nonsense
- **Snowball:** Improvement on Porter, multilingual support

Lemmatization:

```

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

# Without POS tag – defaults to noun
lemmatizer.lemmatize("better")           # → "better" (thinks it's a noun)

# With POS tag – much better
lemmatizer.lemmatize("better", pos="a")   # → "good"   (adjective)
lemmatizer.lemmatize("running", pos="v")  # → "run"    (verb)
lemmatizer.lemmatize("flies", pos="n")    # → "fly"    (noun)

```

Viva question: Why is lemmatization more accurate than stemming? It uses WordNet (a dictionary of English) to find the real root word, so "better" becomes "good", not "betterr". Stemming just chops off endings based on rules.

POS Tagging:

```

from nltk import pos_tag
tagged = pos_tag(word_tokenize("The cat sat on the mat"))
# → [("The", "DT"), ("cat", "NN"), ("sat", "VBD"), ("on", "IN"), ("the", "DT"),
     ("mat", "NN")]

```

Universal POS tags (simplified set):

```

from nltk.tag import pos_tag_sents # Tag multiple sentences
from nltk.tag import map_tag       # Convert to universal tagset

tagged = pos_tag(tokens, tagset="universal")
# → [("The", "DET"), ("cat", "NOUN"), ("sat", "VERB"), ...]

```

Universal tags (12 tags) vs Penn Treebank tags (45+ tags). Universal is simpler — NOUN, VERB, ADJ, ADV, ADP, DET, PRON, CONJ, NUM, PRT, X, .

N-grams:

```

from nltk import ngrams, bigrams

text = "I love NLP and Python"
tokens = word_tokenize(text.lower())
list(bigrams(tokens))           # → [("i", "love"), ("love", "nlp"), ("nlp", "and"),
                                   ("and", "python")]
list(ngrams(tokens, 3))         # → [("i", "love", "nlp"), ("love", "nlp", "and"),
                                   ...]

```

Frequency Distribution:

```
from nltk import FreqDist
fdist = FreqDist(tokens)
fdist.most_common(10)      # Top 10 most frequent words
fdist["nlp"]               # Count of "nlp" in the text
fdist.plot(20)             # Bar chart of top 20 words
```

Concordance — Word in Context:

```
from nltk.text import Text
text_obj = Text(tokens)
text_obj.concordance("nlp") # Shows "nlp" with surrounding context words
```

re (import re)

Purpose: Regular expressions — pattern matching in strings. Essential for text cleaning.

Key Functions:

Function	What it does	Example
<code>re.findall(pattern, text)</code>	Find ALL matches, return list	<code>re.findall(r"\d+", "abc123def456")</code> → ["123", "456"]
<code>re.search(pattern, text)</code>	Find FIRST match, return match object	Checks if pattern exists
<code>re.match(pattern, text)</code>	Match at START of string only	Returns match object or None
<code>re.sub(pattern, repl, text)</code>	Replace matches with <code>repl</code>	Clean text, remove punctuation
<code>re.split(pattern, text)</code>	Split text at pattern matches	Like <code>str.split()</code> but with regex
<code>re.compile(pattern)</code>	Compile pattern (faster for repeated use)	<code>pattern = re.compile(r"\d+")</code>

Common Patterns for Viva:

```
# Character classes
r"\d"    # One digit (0-9)
r"\D"    # One non-digit
r"\w"    # One word character (letter, digit, underscore)
r"\W"    # One non-word character
r"\s"    # One whitespace (space, tab, newline)
r"\S"    # One non-whitespace
```



```

r"."      # Any character except newline

# Quantifiers
r"abc*"    # "ab" followed by zero or more "c"
r"abc+"    # "ab" followed by one or more "c"
r"abc?"    # "ab" followed by zero or one "c"
r"abc{2}"   # "ab" followed by exactly 2 "c"
r"abc{2,}"  # "ab" followed by 2 or more "c"
r"\d{10}"   # Exactly 10 digits (phone number)

# Anchors
r"^abc"     # Starts with "abc"
r"abc$"     # Ends with "abc"

# Sets
r"[aeiou]"  # Any vowel
r"[^aeiou]" # NOT a vowel
r"[a-zA-Z]" # Any letter
r"[0-9]"    # Any digit

# Groups
r"(abc)+"   # One or more occurrences of "abc"
r"(cat|dog)" # Either "cat" or "dog"

# Common text cleaning patterns
r"^[^a-zA-Z\s]" # Remove anything that's not a letter or space
r"\s+"          # One or more whitespace (replace with single space)
r"^[^\w\s]"     # Remove punctuation (keep words and spaces)
r"<[^>]+>"     # HTML tags
r"\b\w{1,2}\b"  # Words of length 1 or 2 (short words)

```

Raw strings (r"...") — **Viva question:** Why **r**? In normal strings, **\n** is a newline. In **r"\n"**, it's literally backslash-n. Regex uses lots of backslashes — raw strings avoid confusion.

re.findall vs **re.search** vs **re.match**:

```

text = "My email is john@email.com and support@test.org"

# findall — ALL matches → ["john@email.com", "support@test.org"]
emails = re.findall(r"\S+@\S+", text)

# search — FIRST match → match object (has .group(), .start(), .end())
match = re.search(r"\S+@\S+", text)
print(match.group())      # → "john@email.com"

# match — match at START ONLY
match2 = re.match(r"\S+@\S+", text)
print(match2)             # → None (text doesn't START with email)

```

Flags:

```
re.findall(r"abc", text, re.IGNORECASE)    # Case-insensitive
re.findall(r"^abc", text, re.MULTILINE)    # ^ matches start of each line
re.findall(r"abc.def", text, re.DOTALL)    # . matches newline too
```

scipy (`import scipy`)

Purpose: Scientific computing — advanced statistics, optimization, signal processing.

What's used in practicals:

```
from scipy import stats

# Normal distribution – probability density function (used in Naive Bayes)
stats.norm.pdf(x, loc=mean, scale=std)    # Probability density at x
stats.norm.cdf(x, loc=mean, scale=std)    # Cumulative probability P(X ≤ x)
stats.norm.ppf(q, loc=mean, scale=std)    # Inverse CDF (quantile)

# Statistical tests
stats.ttest_ind(group1, group2)           # Independent t-test (compare two
groups)
stats.shapiro(data)                       # Shapiro-Wilk test for normality
stats.skew(data)                           # Skewness
stats.kurtosis(data)                       # Kurtosis (peakedness)
stats.describe(data)                       # Count, mean, std, min, max, skew,
kurtosis

# Z-score
stats.zscore(data)                         # Same as StandardScaler, but for
numpy arrays
```

Viva question: Difference between `stats.norm.pdf` and `stats.norm.cdf`?

- `pdf(x)`: Probability density at point x. Height of the bell curve at x. Used in Gaussian Naive Bayes to compute `P(Feature | Class)`.
- `cdf(x)`: Probability that $X \leq x$. Area under the curve from $-\infty$ to x. Used to find percentiles.

Example — GaussianNB uses pdf under the hood:

```
# For a feature value x in class with mean=5, std=1:
likelihood = stats.norm.pdf(x, loc=5, scale=1)
# This is the P(x | Class) that Naive Bayes multiplies
```

Summary Table — Library vs Practical

Library	Practical(s)	Primary Use
pandas	1, 2, 3, 4, 5, 6, 8, 9, 10, 12	Load data, DataFrame ops, cleaning, groupby
numpy	1, 2, 3, 4, 5	log, sqrt, mean, std, array ops
matplotlib	8, 9, 10	Plot customization, subplots, saving figures
seaborn	8, 9, 10	Histogram, boxplot, countplot, pairplot, heatmap
sklearn.preprocessing	1, 5, 12	StandardScaler, MinMaxScaler, LabelEncoder
sklearn.model_selection	4, 5, 6, 12	train_test_split, cross_val_score
sklearn.linear_model	4, 5	LinearRegression, LogisticRegression
sklearn.naive_bayes	6	GaussianNB
sklearn.svm	12	SVC (linear, poly, rbf)
sklearn.metrics	4, 5, 6, 12	MAE, MSE, R ² , confusion_matrix, classification_report
sklearn.feature_extraction	7	TfidfVectorizer, CountVectorizer
nltk	7	Tokenization, stop words, stemming, lemmatization, POS
re	7	Text cleaning, pattern matching
scipy	6 (indirectly)	Statistical distributions (used by GaussianNB)

Common Python Built-ins Used in Practicals

```

len(list)           # Length of list
range(start, stop, step) # Generate number sequence
enumerate(list)     # Get (index, value) pairs
zip(list1, list2)   # Pair up elements from two lists
sorted(list)        # Return sorted copy
reversed(list)      # Reverse iterator
type(obj)           # Check type: type(df) → pandas.DataFrame
isinstance(obj, cls) # Type check: isinstance(df, pd.DataFrame)
map(func, iterable) # Apply function to every element
filter(func, iterable) # Filter elements based on function
open("file.txt", "r") # Read/write files
with open("f.txt") as f: # Context manager – auto-closes file
    content = f.read()

```

List comprehensions (used everywhere):

```
[x**2 for x in range(10)]           # [0, 1, 4, 9, ...]  
[w for w in words if w not in stop_words] # Filter stop words  
[df["col"].iloc[i] for i in indices]    # Select by index list
```